

Разработка высоконагруженной Event-driven микро сервисной платформы Sport Point для городских спортивных ивентов

Шабанов Максим

Тема проекта

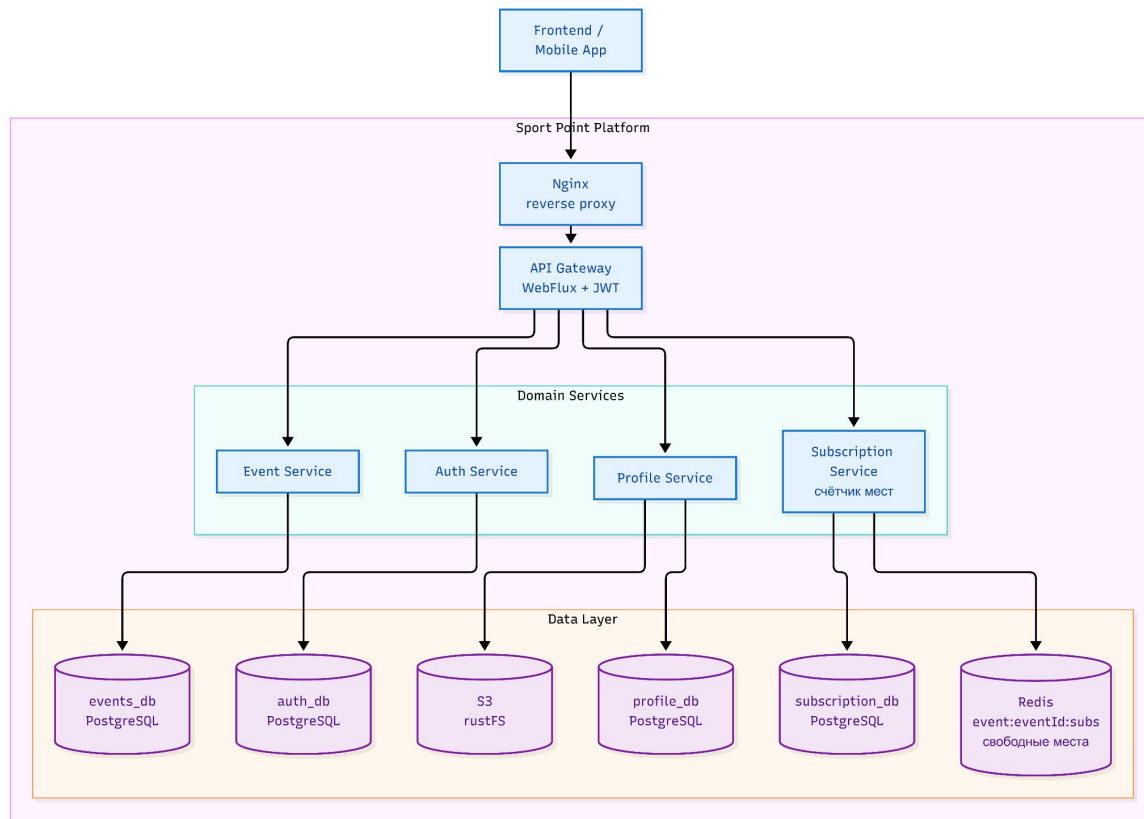
Sport Point - это платформа, которая объединяет активных людей города.

Основные возможности:

- Карта ивентов - пользователи видят все мероприятия на интерактивной карте города с возможностью фильтрации
- Создание ивентов - любой пользователь может организовать событие (тренировка, соревнование и т.д.)
- Подписка и участие - запись на ивент и просмотр списка участников
- Профиль пользователя - каждый может рассказать о себе, своих интересах и спортивной активности

Стек технологий

Категория	Технологии
Языки и фреймворки	Java, Spring Boot, Spring WebFlux, TypeScript, React
Работа с данными	Hibernate, Spring Data JPA, Spring Data Redis, Spring Kafka
Базы данных и кэш	PostgreSQL, Redis, S3 (rustFS)
Брокер сообщений	Apache Kafka
Инфраструктура	VPS, Docker, Nginx
Безопасность	Spring Security, JWT
Системы сборки	Maven, Vite
UI библиотеки	Ant Design, Leaflet



Структура проекта

Архитектура

Архитектура

Платформа построена на принципах Event-Driven Architecture.

Всё строится вокруг событий:

- Взаимодействие - сервисы общаются только через события, без REST-запросов между собой
- Данные - Database-per-Service и перенос состояния через события ECST (Event-Carried State Transfer)
- Чтение - отдельные проекции по принципу CQRS (Command Query Responsibility Segregation)
- Операции - распределённые сценарии по принципу Saga (хореография)

Взаимодействие

Сервисы общаются только через события:

- Принципиально без REST-запросов между сервисами: каждый сервис публикует события и подписывается на чужие, не зная о существовании друг друга.
- Синхронный вход, асинхронный «хвост» - клиент получает мгновенный ответ от сервиса, которому адресован запрос; каскадные операции (создание профиля, подписок) выполняются в фоне при помощи Apache Kafka
- Слабая связанность - сервисы не знают друг о друге и не зависят от доступности соседей в момент обработки запроса: падение одного сервиса не блокирует остальные

Данные

Каждый сервис владеет своими данными:

- Database-per-Service — каждый сервис работает со своей базой данных; базы не шарятся между сервисами, и получить чужие данные можно только через события
- Event-Carried State Transfer (ECST) — событие несёт не только факт произошедшего, но и само состояние: получатель может обновить свои данные без обратного запроса к источнику
- Только иммутабельные поля — денормализуются лишь неизменяемые данные; мутабельные не дублируются, чтобы избежать рассинхрона между сервисами

Чтение и операции

Проекции для чтения и распределённые операции:

- CQRS (Command Query Responsibility Segregation) - запись и чтение разделены: на основе событий строятся отдельные read-модели (проекции), которые обновляются асинхронно и отдаются UI
- Saga (хореография) — распределённые бизнес-операции выполняются цепочкой событий, без центрального оркестратора: каждый сервис реагирует на события и публикует свои

Подписки - отдельный сервис

Почему подписки были выделены в отдельный сервис, а не как часть сервиса ивентов:

- Подписки — самое конкурентное место в системе: на один ивент могут одновременно записываться десятки и сотни пользователей, а мест ограниченное количество.
- Учёт мест в реляционной базе под такой нагрузкой приводит к блокировкам строк, очереди ожидания и падению пропускной способности.

Решение:

- Свободные места учитываются атомарным счётчиком в Redis, а сервис подписок синхронно работает с ним при обработке событий.
- Атомарность операции обеспечивает сам Redis, поэтому блокировки строк и очередь ожидания не возникают в принципе.

Что даёт:

- Сервис подписок масштабируется независимо от остальных и имеет собственную базу данных.
- Падение сервиса подписок не влияет на создание и чтение ивентов — остальная система продолжает работать.

Обеспечение отказоустойчивости

Обеспечение отказоустойчивости

Для отказоустойчивости применяется связка архитектурных паттернов:

- Transactional Outbox
- Consumer manual commit
- Идемпотентность (Inbox pattern)
- Retry topic
- DLT (Dead Letter Topic)

Transactional Outbox

Проблема: нужно атомарно записать изменение в БД и опубликовать событие в Kafka. Если сделать два действия подряд - можно упасть между ними: в БД записали, в Kafka не отправили (или наоборот).

Решение: в той же транзакции, что и изменение домена, пишем событие в отдельную таблицу outbox. Отдельный планировщик (scheduler) читает outbox и публикует в Kafka, после чего помечает запись отправленной.

Итог: Outbox (на стороне продюсера) — гарантирует, что событие уйдёт.

Consumer manual commit

Проблема: если Kafka-consumer коммитит offset автоматически, он может подтвердить сообщение до того, как реально его обработал. Падение в этот момент - сообщение потеряно.

Решение: отключаем auto-commit и коммитим offset вручную, только после успешной обработки (в идеале - в той же транзакции, что и запись в БД).

Итог: “обработали - подтвердили”, а не наоборот. Основа для at-least-once + идемпотентности.

Идемпотентность (Inbox pattern)

Проблема: at-least-once (следствие outbox + manual commit) означает, что сообщение может прийти повторно - при ребалансе или ретрае. Без защиты повторная обработка ломает состояние: двойной DECR, дубли INSERT, повторные ивенты.

Решение: на стороне консьюмера используется Inbox-таблица (симметрично Outbox на стороне продюсера):

- каждое событие несёт заголовок idempotency-key;
- консьюмер пытается вставить ключ в inbox_message (saveAndFlush);
- при дубликате сообщение подтверждается (ack.acknowledge()) и обработка пропускается;
- при отсутствии заголовка — IllegalArgumentException → сразу в DLT.

Итог: Inbox (на стороне консьюмера) - гарантирует, что повторно пришедшее событие не обработается дважды.

Retry topic

Проблема: часть ошибок при обработке события - временные: сервис-получатель недоступен, БД под нагрузкой, таймаут при обращении к внешней системе. Такое сообщение имеет смысл попробовать обработать ещё раз. Но ретраить его в основном топике нельзя - это блокирует партицию и тормозит обработку всех остальных сообщений.

Решение: при ошибке сообщение перекладывается в отдельный retry-топик, а не возвращается в основной поток. Ретраи выполняются с задержкой, число попыток ограничено. Топики ретраев нумеруются по индексу попытки, что делает поведение системы наблюдаемым.

Итог: временные сбои переживаются без потери сообщения и без блокировки основного потока. Нагрузка на упавший сервис растёт постепенно, а не лавинообразно.

DLT (Dead Letter Topic)

Проблема: часть ошибок - неретраибельные по своей природе: битый формат сообщения, отсутствие обязательного поля или заголовка, логическая ошибка в данных. Ретраи таких сообщений бессмысленны и вредны: они забивают топики и маскируют реальную проблему.

Решение: ошибки явно классифицируются на ретраибельные и неретраибельные. Неретраибельные исключения не попадают в retry-цикл и сразу направляются в DLT - отдельный топик для “мёртвых” сообщений. Туда же попадает всё, что исчерпало лимит попыток в retry-топике. Сообщения в DLT не теряются: их можно разобрать, исправить и переиграть.

Итог: “мёртвые” сообщения не блокируют систему и не теряются бесследно. По содержимому DLT видно, что именно и почему не обработалось, — это точка диагностики и ручного вмешательства.

Схемы взаимодействия

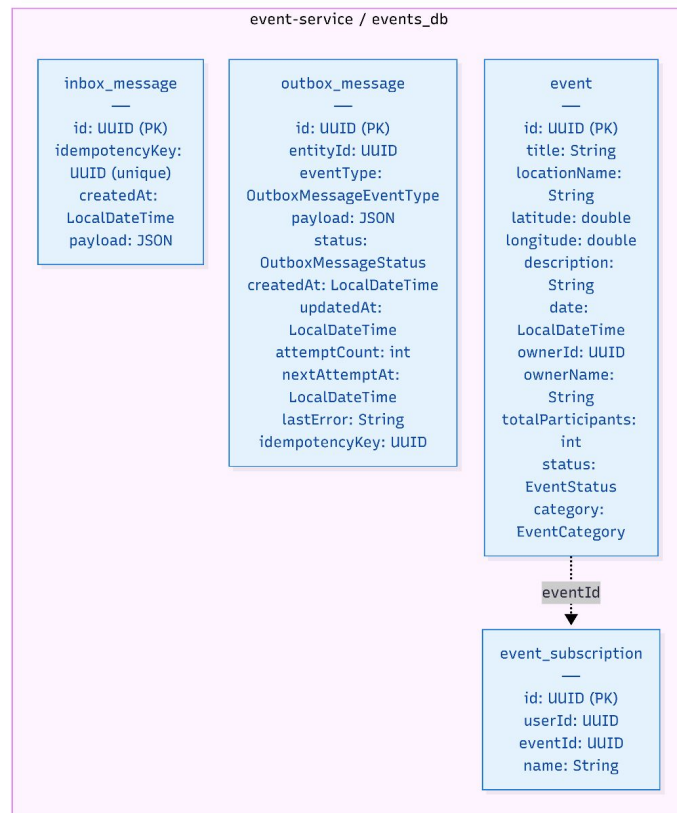
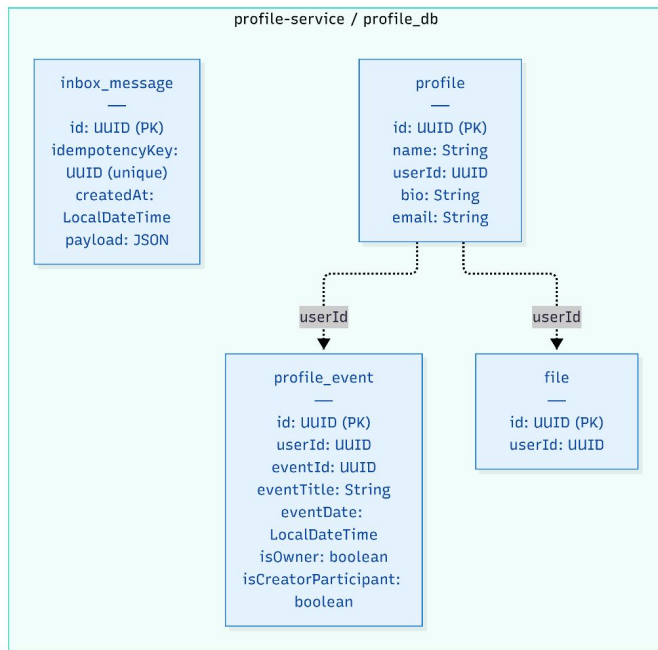


Схема базы данных часть 1

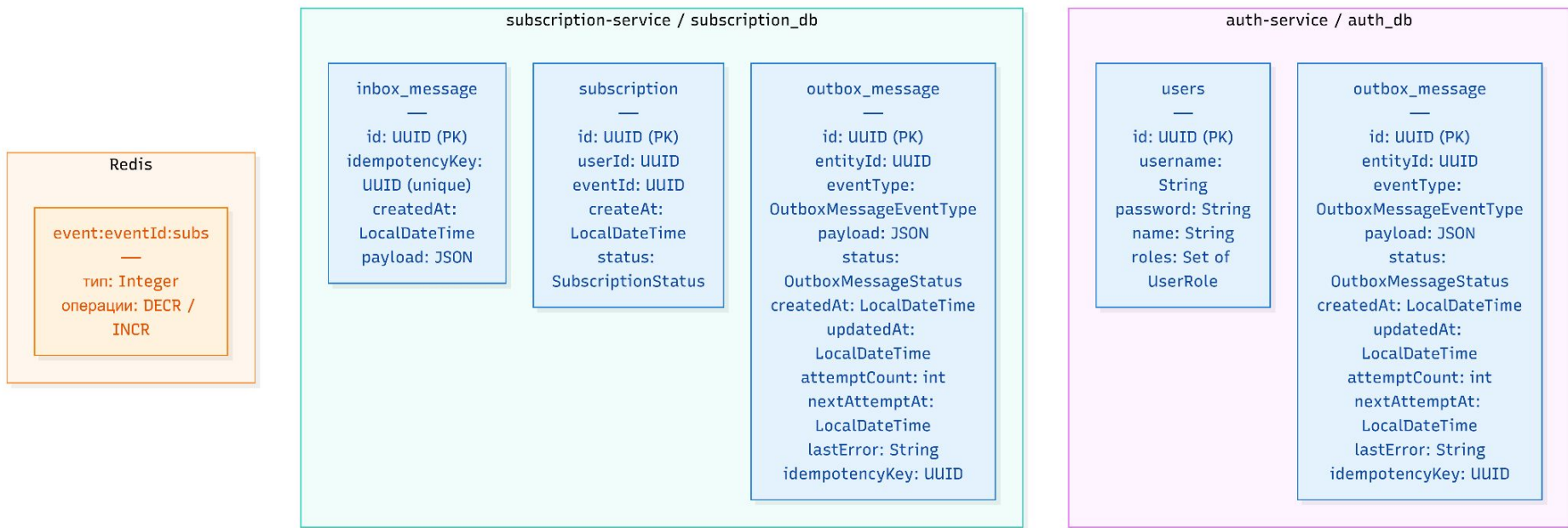


Схема базы данных часть 2

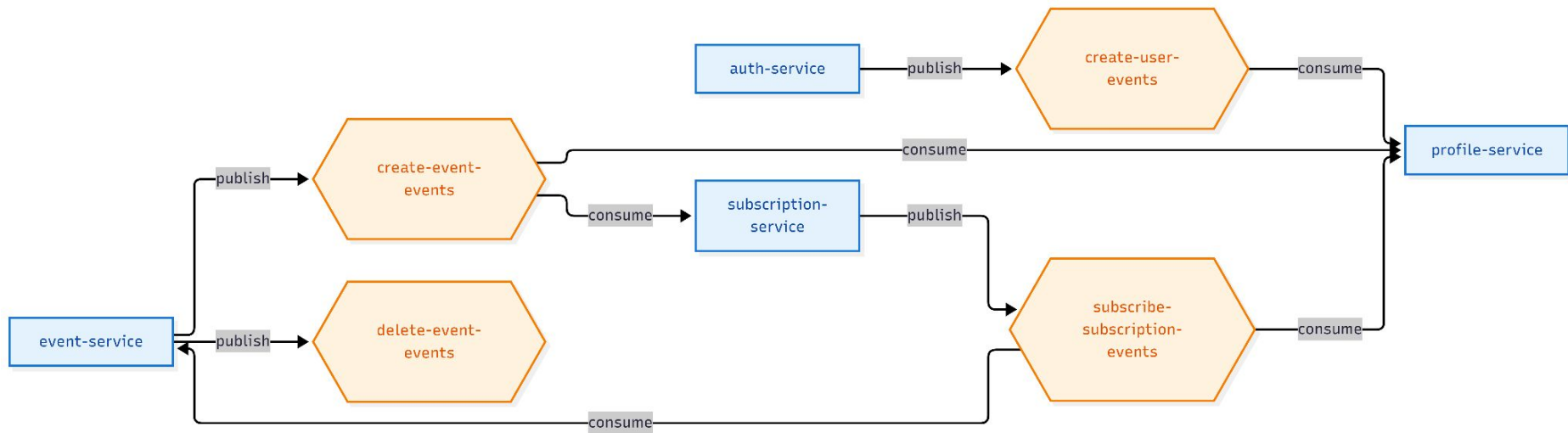
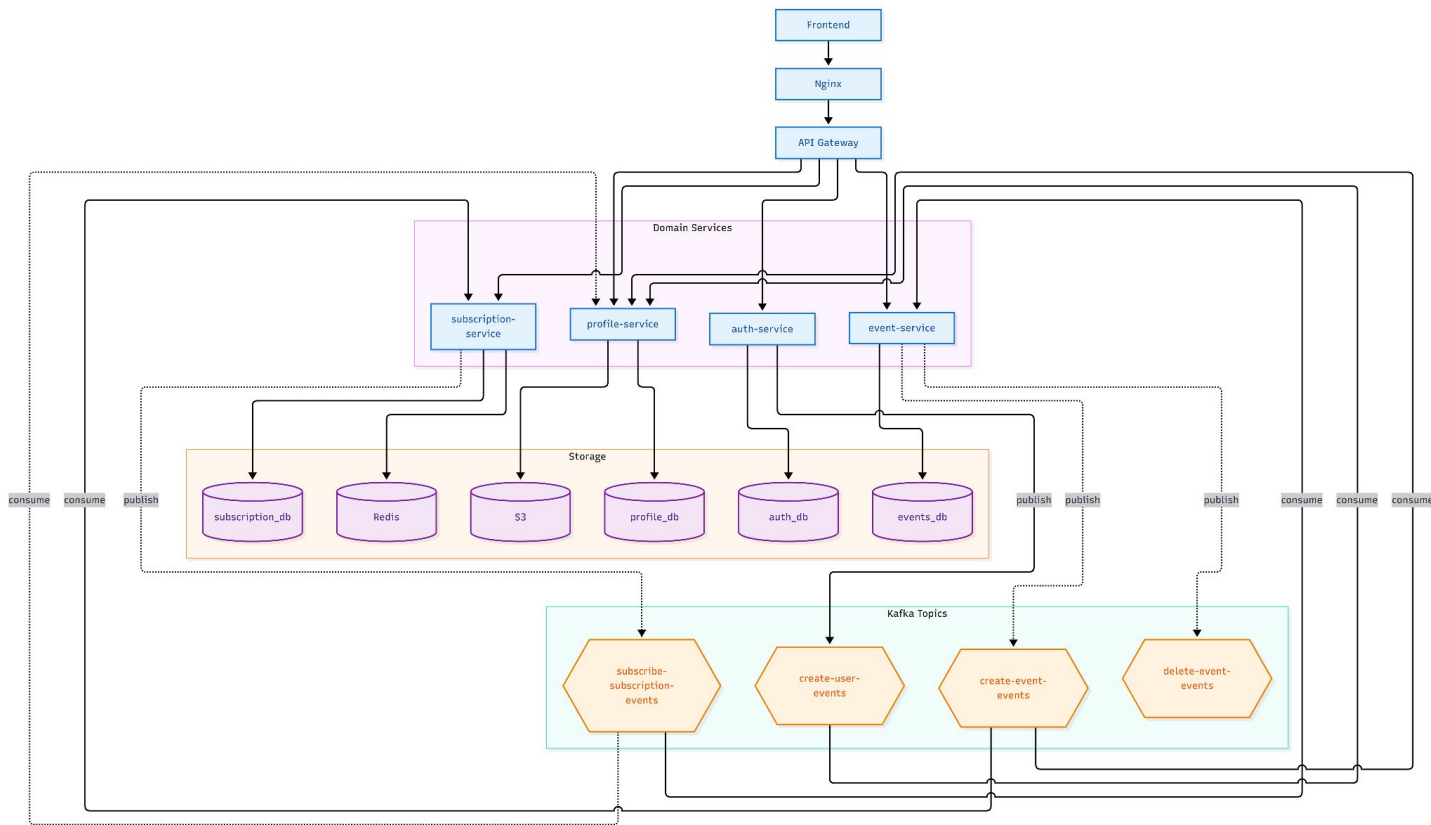
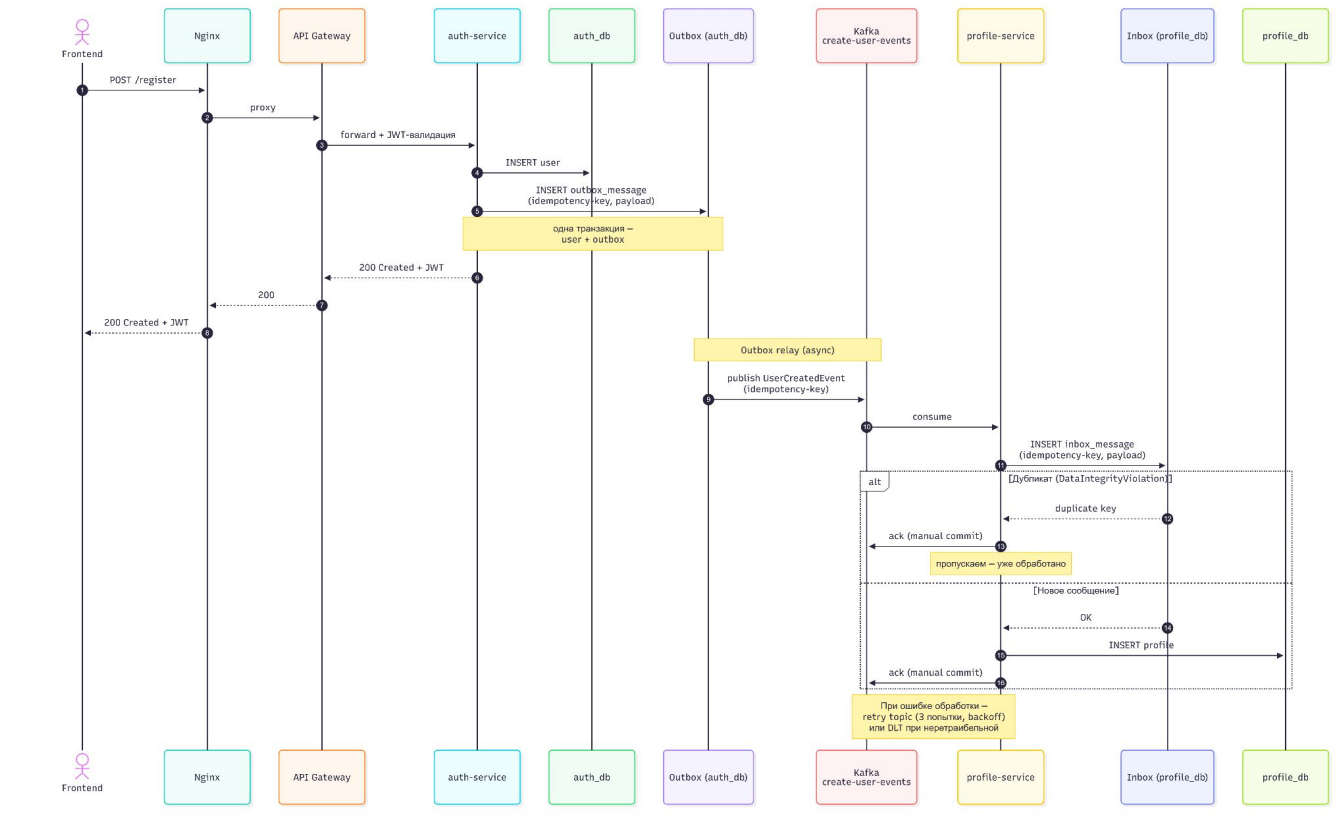


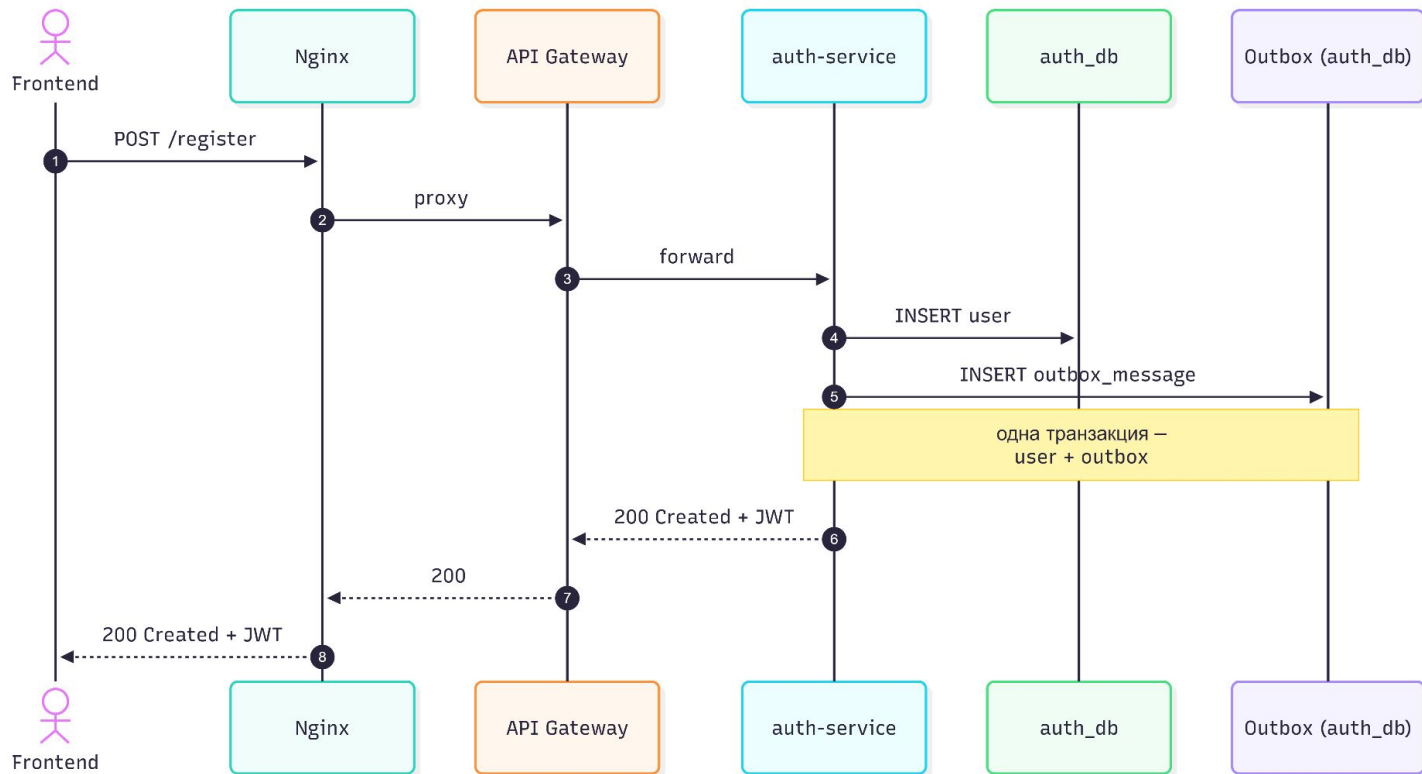
Схема взаимодействия между сервисами



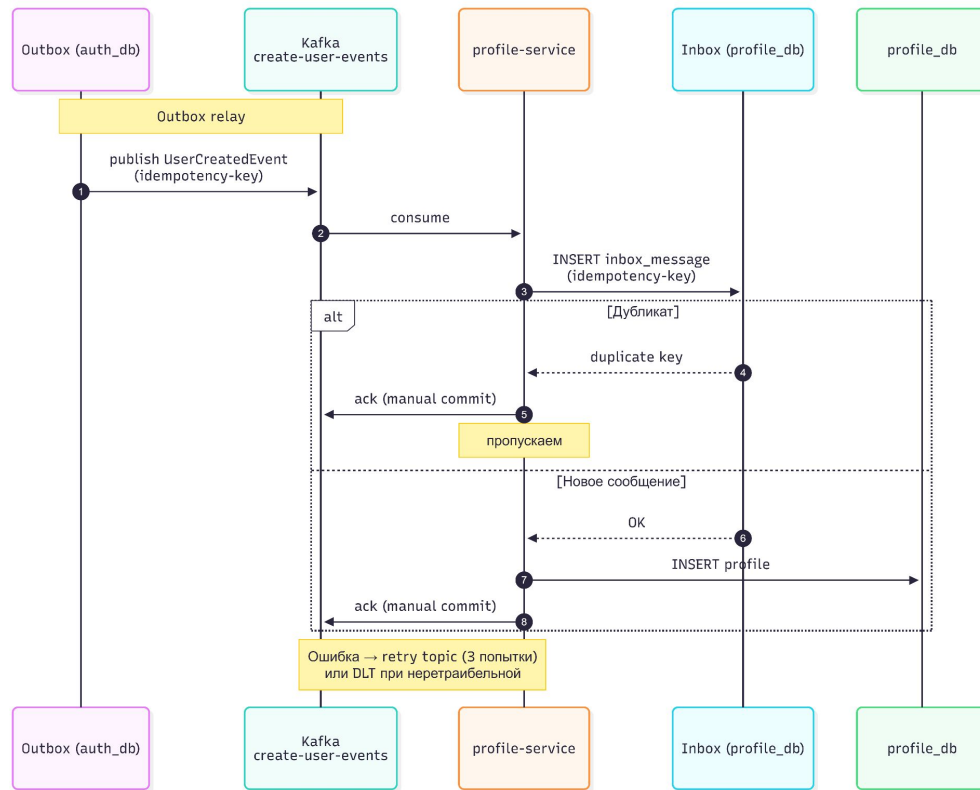
Общая схем авзаимодействия



Use-case Регистрация с созданием профиля



Use-case Регистрация с созданием профиля - Синхронный ответ



Use-case Регистрация с созданием профиля - асинхронный «хвост»